

Midterm Review

CS 180

Scope

- Everything discussed in lecture up to and including stereo (no epipolar questions)
- Key concepts and coding knowledge from Project 0, Project 1, Project 2, Project 3A, and Project 3B

How do I study for this?

- Discussion sheets have similar style questions
- Quizzes (answers released on Ed)
- Understand the projects well

You have a cheat sheet! so don't memorize formulas

Agenda

Structured with homeworks!

- **Project 0:** Pinhole camera model
- **Project 1:** Gaussian / laplacian pyramids
- **Project 2:** Filters + frequencies
- **Project 3A + stereo:** Homography + RANSAC

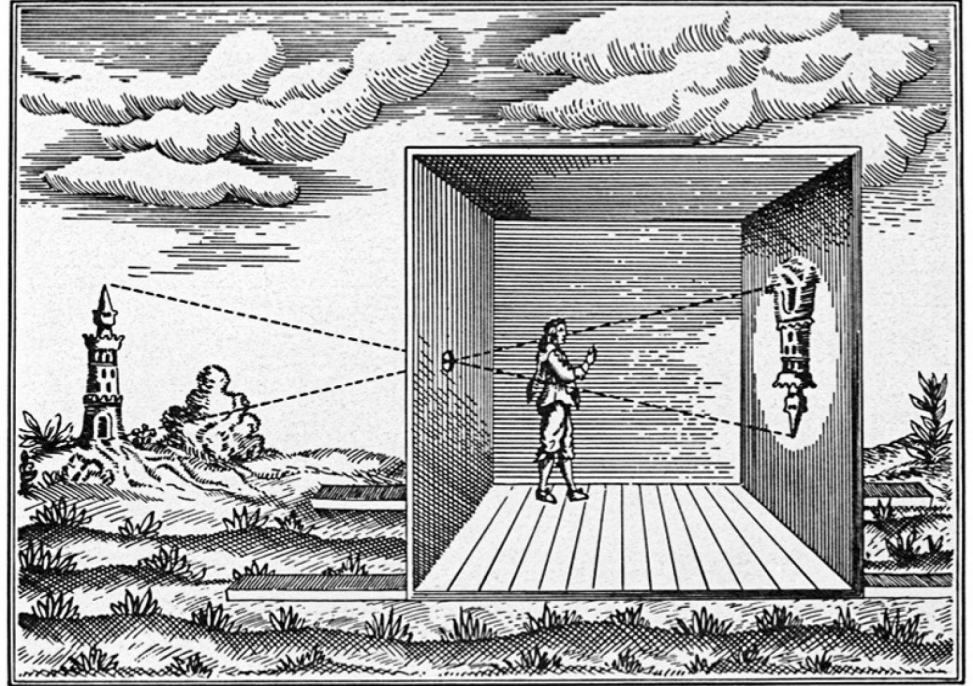
Which one you want to go over first? 🖐️

Intermediate checkpoint questions to discuss with your neighbors throughout

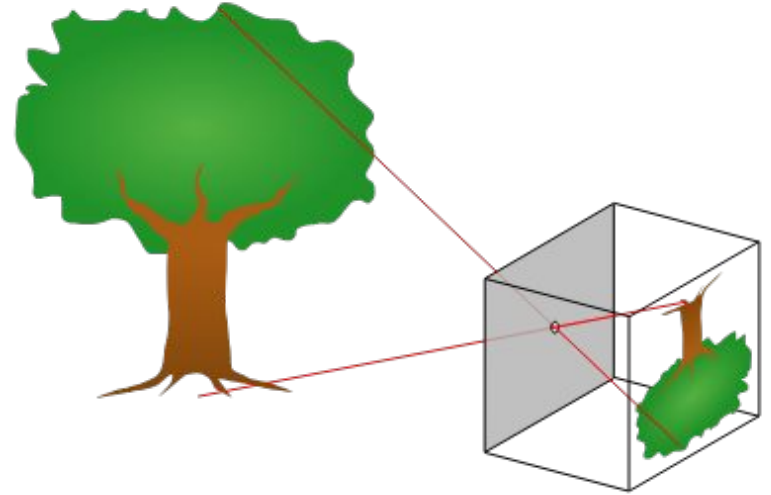
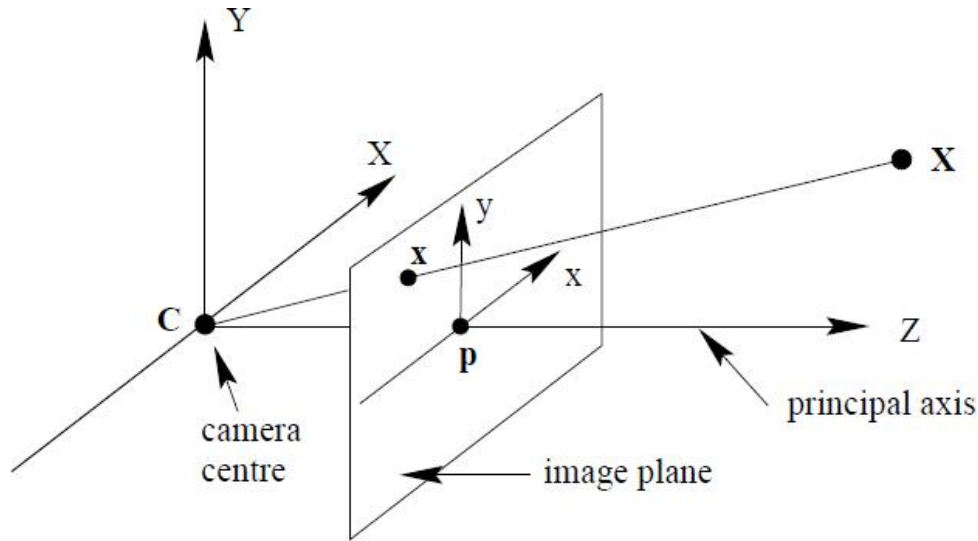
Project 0

Key Topics:

- Pinhole Camera Model



Pinhole Camera Model



- The object on the image plane should be inverted! But for our convenience, we can pretend the image plane is in front of the camera center

Concept question!

- How is the position of a point in the 3D world connected to its position on the image plane?
- T/F? The "focal" in focal length refers to the distance at which things are in focus

Concept question!

- How is the position of a point in the 3D world connected to its position on the image plane?

Via a ray through the pinhole, which results in:

- Similar triangles + pinhole projection.
- $(x / X) = (f / D)$

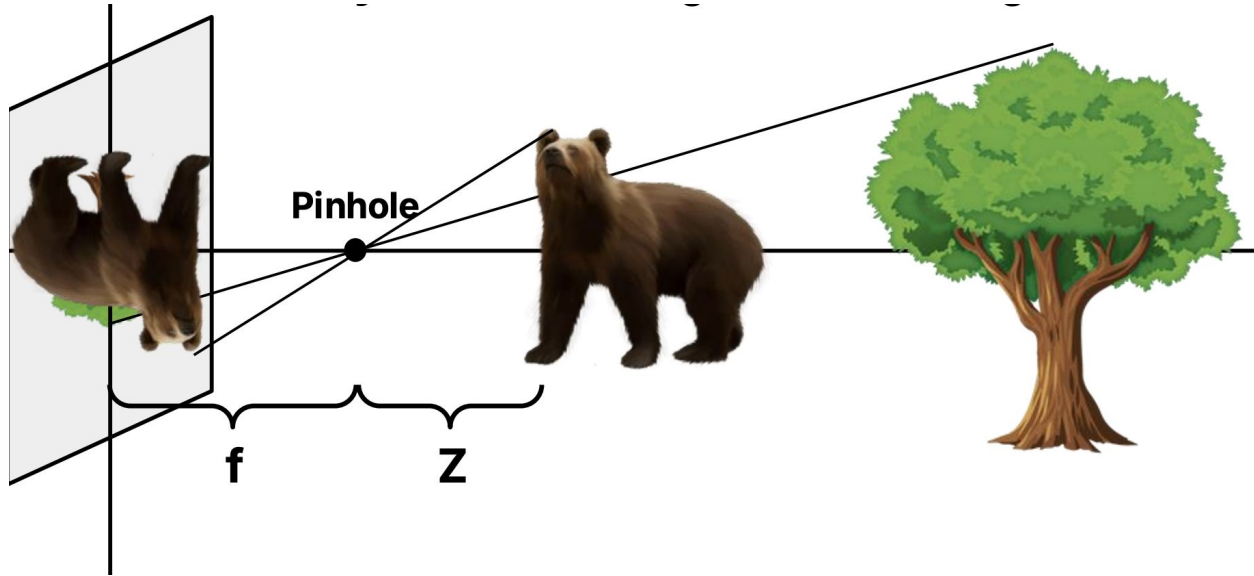
x: projected size, **X**: size in world

f: focal length, **D**: distance to pinhole (depth)

T/F? The “focal” in focal length refers to the distance at which things are in focus

False! Unrelated to blurriness

Dolly Zoom (+ questions)



- What happens to the tree and the bear if we zoom out (smaller f)?
- What happens to the tree and the bear if we walk in (smaller Z)?
- What happens to the tree and the bear if we zoom out and walks in at the same time?

Project 1



Aliasing and Antialiasing

Gaussian Pyramids

Multi-Scale Image Alignment

Sanity check!

T/F: An image in a computer always has shape (H, W, 3)

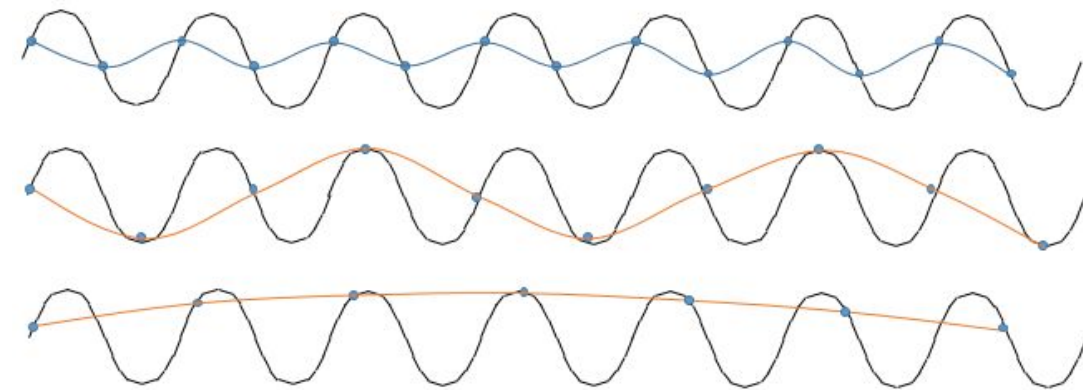
Sanity check!

T/F: An image in a computer always has shape (H, W, 3)

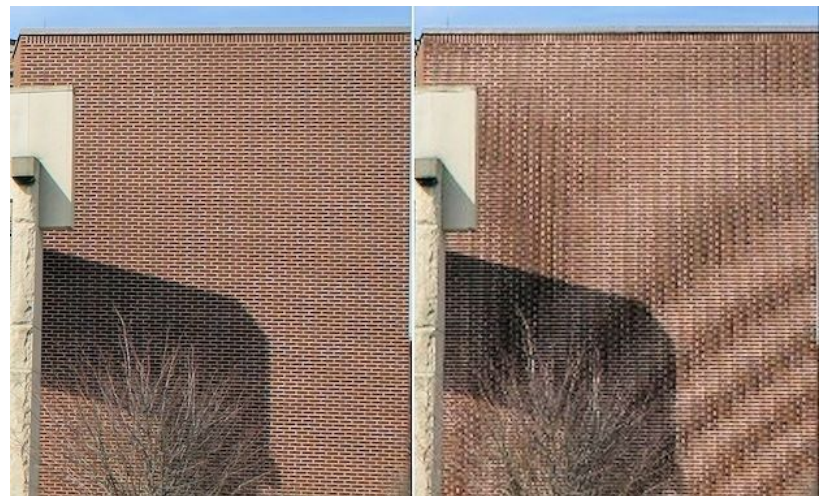
False!

- Can be channel-first (CHW) or channel-last (HWC)
- Can be single-channel (1) e.g., for depth

Aliasing



Aliasing Examples



Concept check!

Given a signal of maximum frequency f_{max} , what is the max frequency if I downsample the image by a factor of 2?

Name a scenario where you could see aliasing 1) with your own eyes and 2) with a camera

Concept check!

Given a signal of maximum frequency f_{max} , what is the minimum sampling frequency required to fully reconstruct the signal?

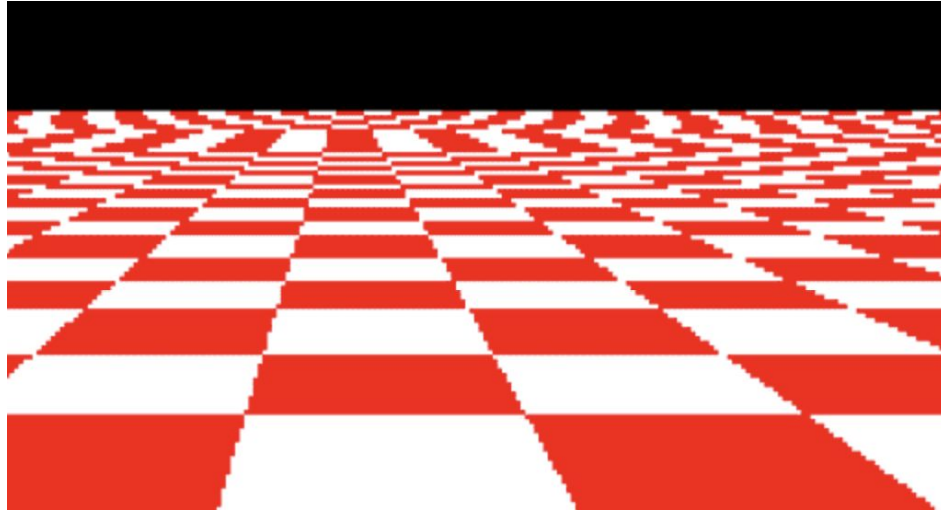
$f_{\text{max}} / 2$

Name a scenario where you could see aliasing 1) with your own eyes and 2) with a camera

- 1) Looking through a screen door
- 2) Taking a picture of a repeating pattern at low resolution

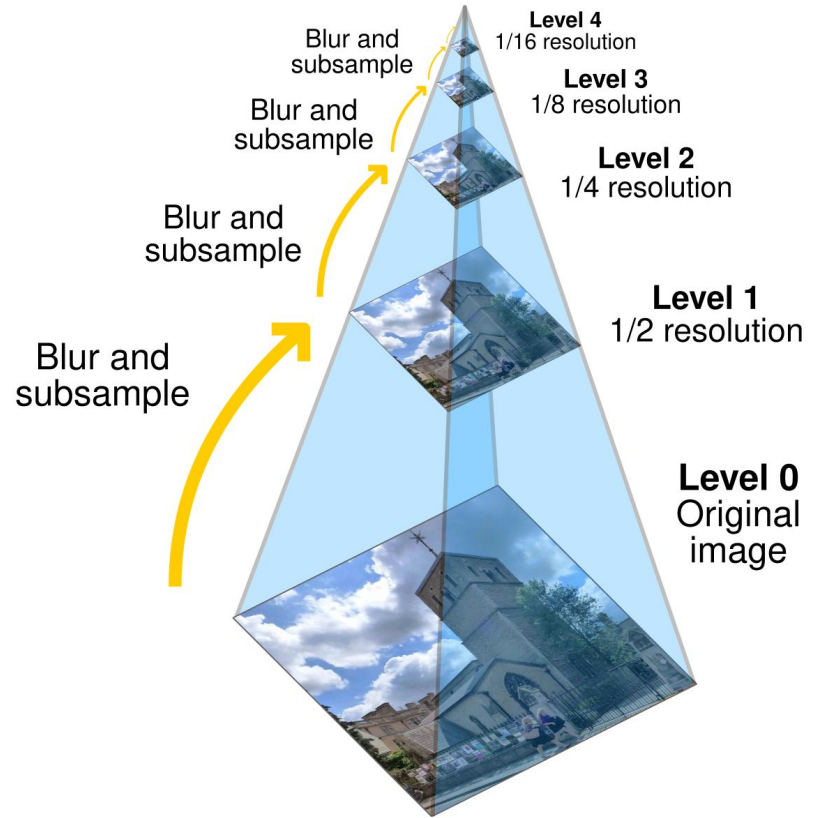
Antialiasing

- How can we prevent aliasing?
 - Increase sampling rate
 - Filter out the high frequencies that can be aliased (blur)



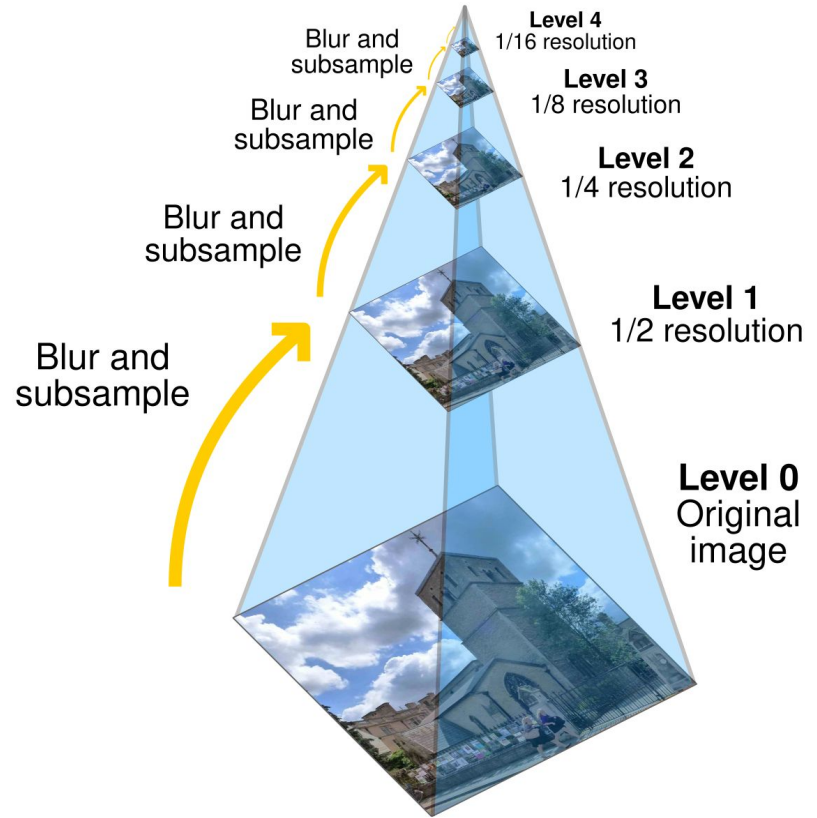
Gaussian Pyramids

- Computationally efficient for image processing
- Start with the full resolution image
- **Repeat:**
 - Apply Gaussian blur to counter aliasing
 - Downsize by half
- Until the minimum resolution is reached



Multi-Scale Image Alignment

1. Create Gaussian pyramid
2. Calculate alignment at the minimum resolution
3. Propagate and refine the alignment at higher resolutions
 - Search a small range around the alignment from the previous resolution



*Potential remove?

Concept Check!

What does a Gaussian pyramid represent in the frequency space?

T/F: blurring an image after sampling is effective for antialiasing

What is the difference between a gaussian *pyramid* and *stack*?

Concept Check!

What does a Gaussian pyramid represent in the frequency space?

A Gaussian pyramid represents multiple levels of low-pass filtering on an image.

T/F: blurring an image after sampling is effective for antialiasing

False, aliasing occurs during sampling.

What is the difference between a gaussian *pyramid* and *stack*?

A stack does not include image resizing, a pyramid does

Project 2

Key Topics:

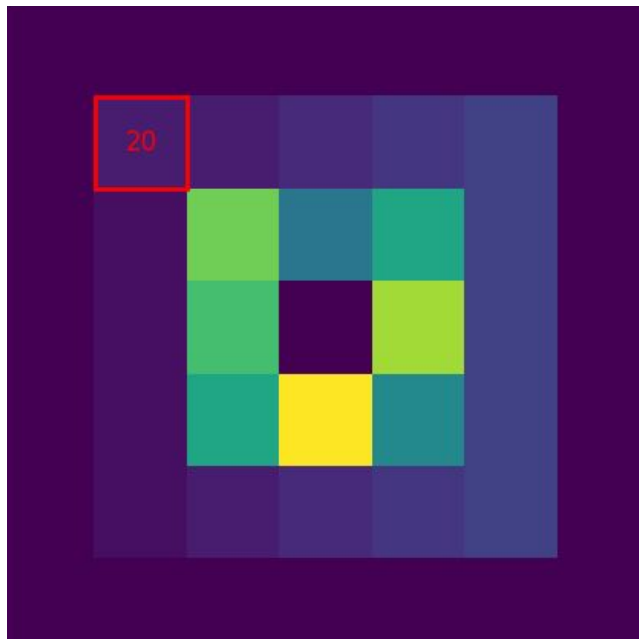
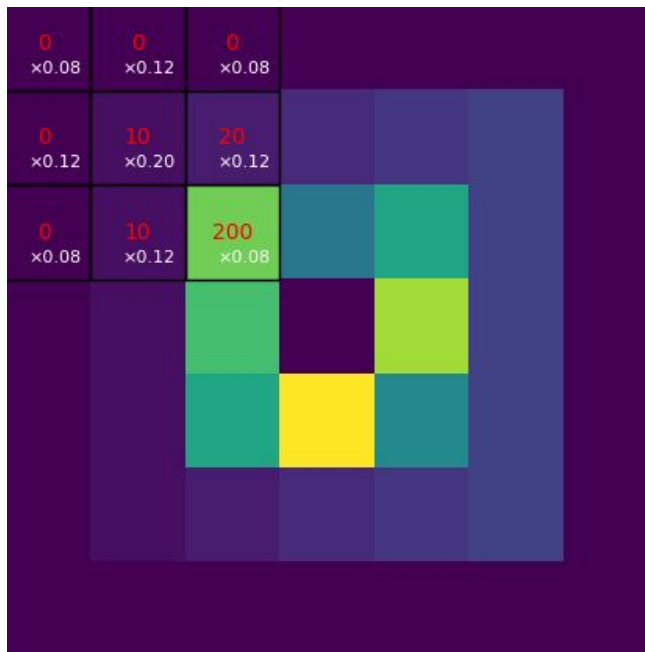
- Convolution
- Filters
 - Low Pass, High Pass, Hybrid Images
- Fourier Transform
- Gaussian and Laplacian Pyramids
 - Image Blending

Convolution

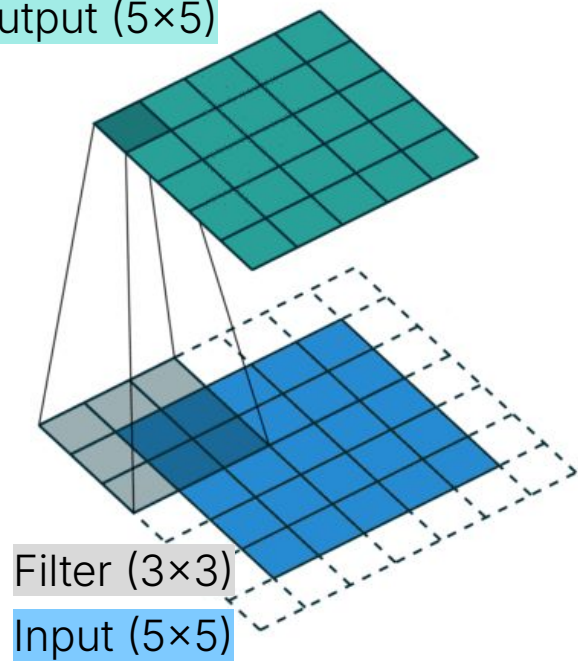
- Image processing technique that can enhance or suppress certain image features, like edges or noise
- **Cross-correlation:** the filter is shifted across the input image
- **Convolution:** the filter is flipped horizontally and vertically before being shifted across the input image.
- Convolution is commutative.
- **Convolution theorem:** convolving = multiplying frequencies

Convolution

Output

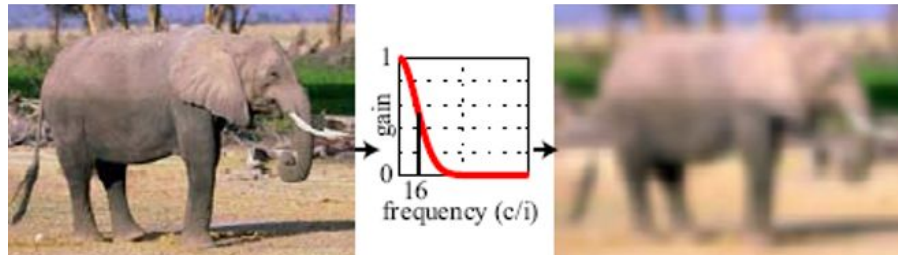


Output (5×5)



Low-Pass Filters

- Filter out high frequencies
- Can be applied to images **before** sampling to prevent aliasing
- **Examples:**
 - **Mean filter:** each pixel value is replaced by a neighborhood average
 - **Gaussian filter:** each pixel value is replaced by a neighborhood average, weighted according to the Gaussian distribution
 - (Cannot do with convolution) Median filter



High-Pass Filters

- Filter out low-frequency components, like flat regions
- Highlight high-frequency components, like fine details or edges
- **Examples:**
 - **Unsharp masking:** increase the contrast along the edges in an image by subtracting a low-pass filtered image from the original

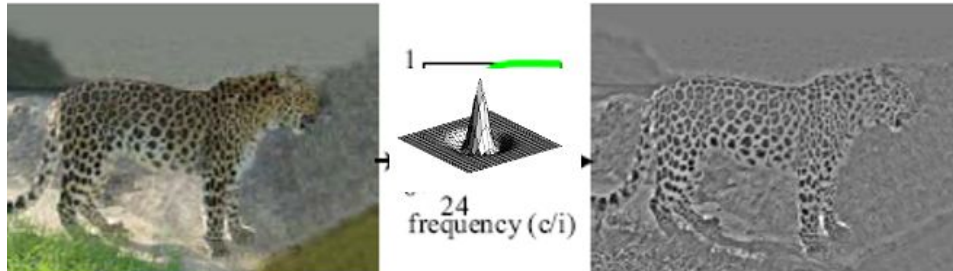
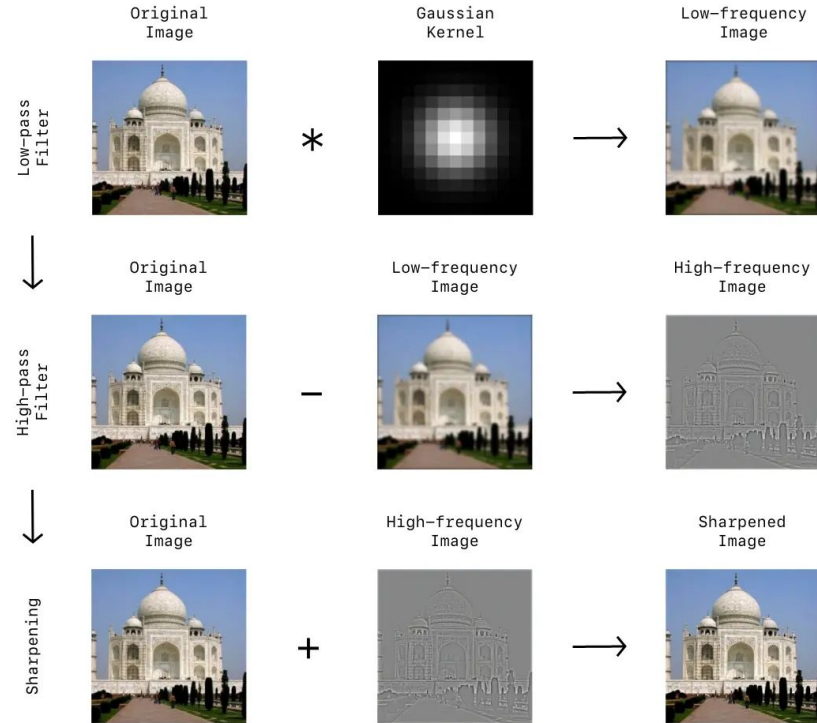


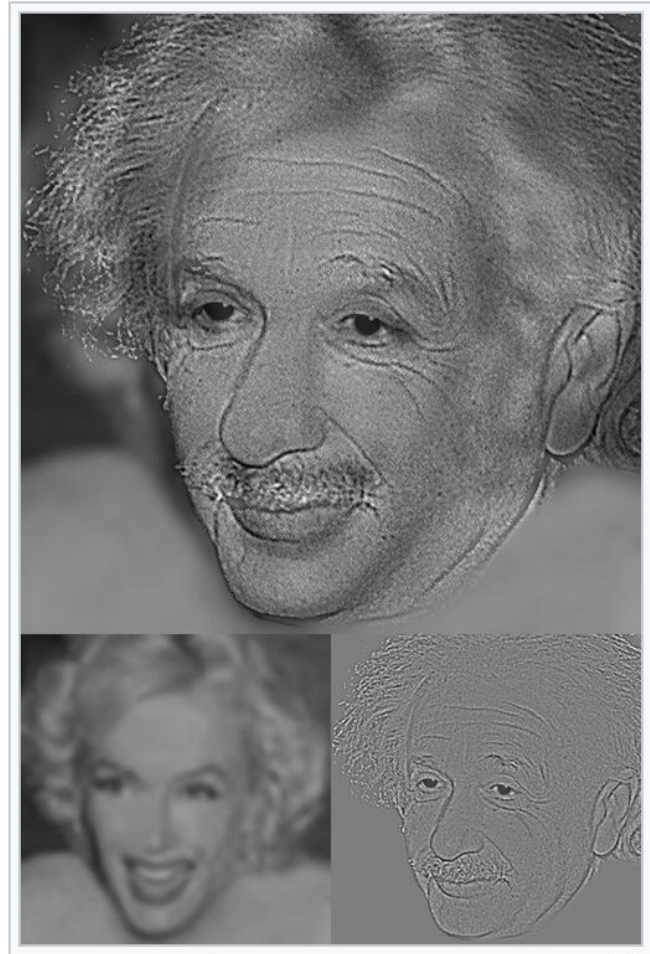
Image Sharpening



Hybrid Images

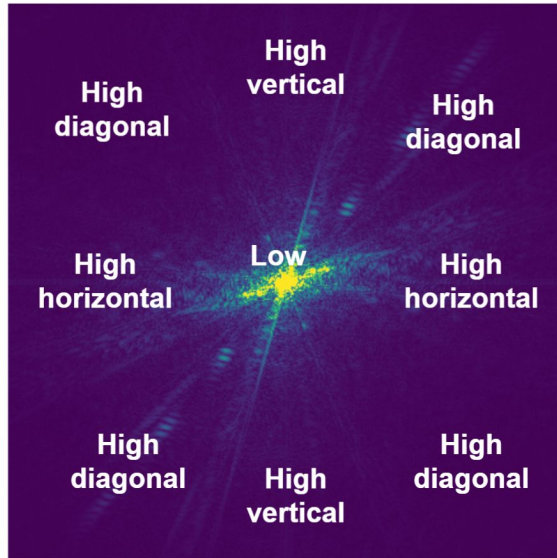
- Hybrid images are constructed by adding the low spatial frequency components of one image with the high spatial frequency components of another image

$$\text{Hybrid} = \text{LowPass}(\text{im1}) + \text{HighPass}(\text{im2})$$

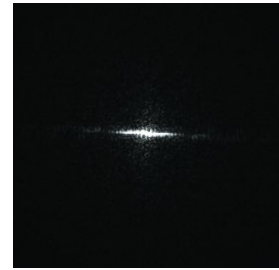
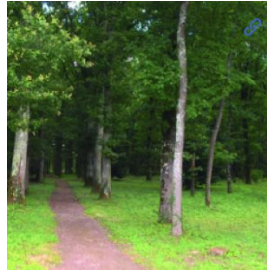
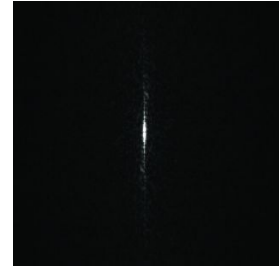


Fourier Transform

- Maps a signal from the **spatial** domain to the **frequency** domain
- Describes the extent to which different frequencies are present
- Maps signals to an **amplitude** and **phase**

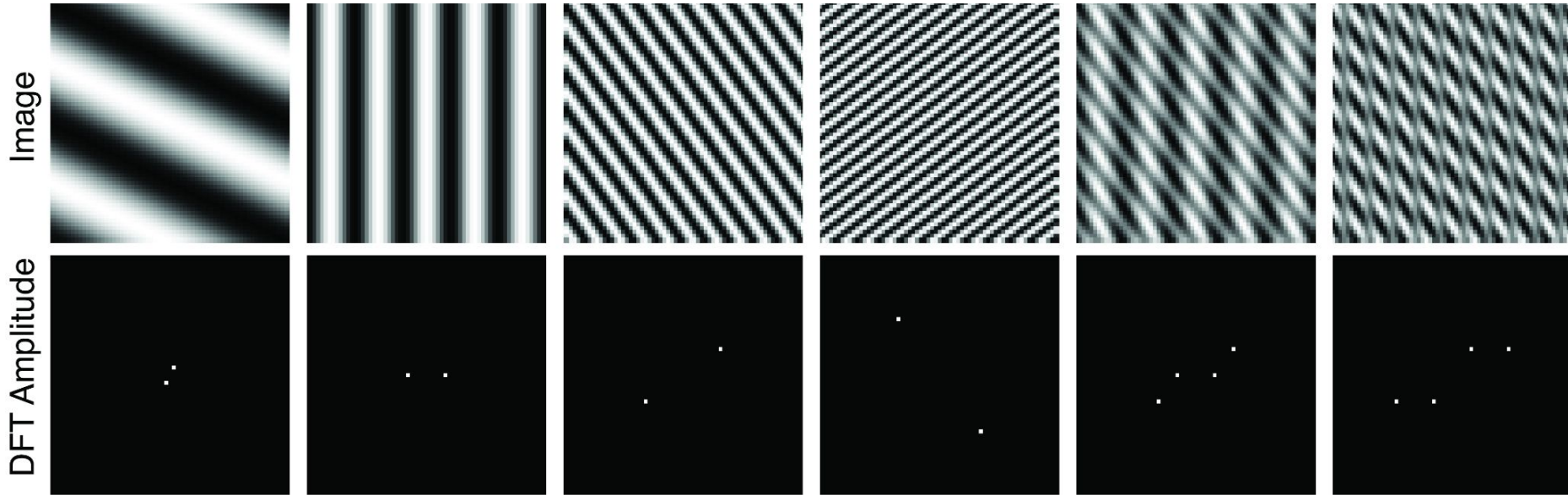


Amplitude



Amplitude plot in 2D

Each FFT "pixel": amplitude at certain frequency and angle

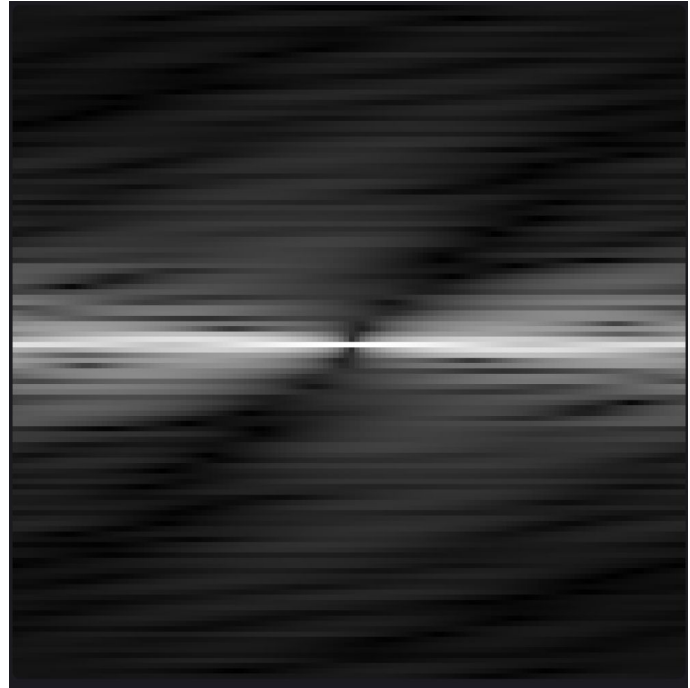
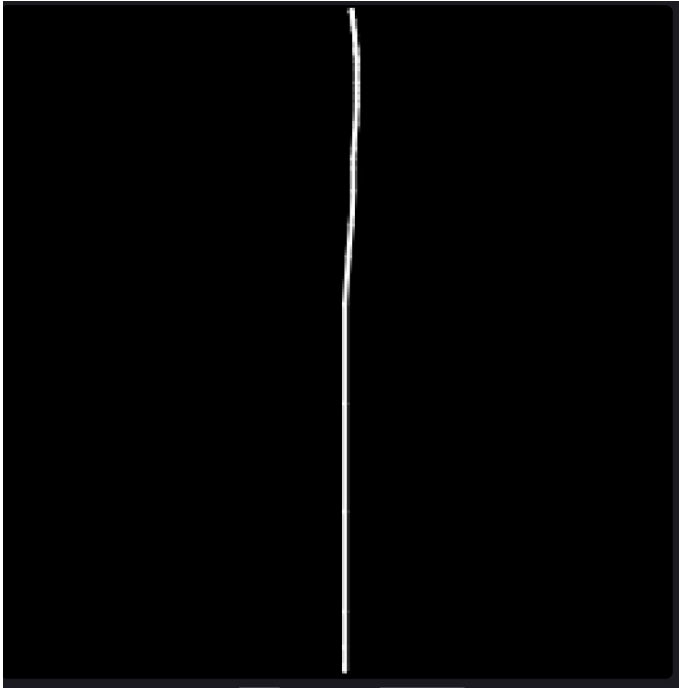


Sanity check

A vertical line in an image gets mapped to a __ line in FFT space?

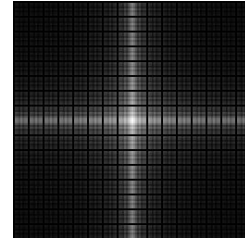
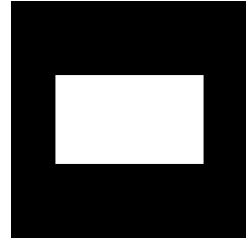
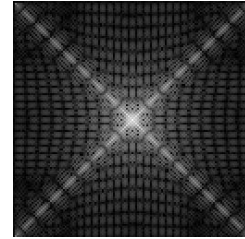
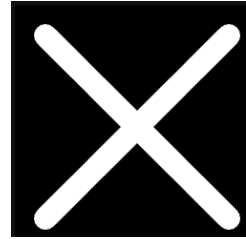
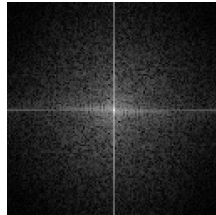
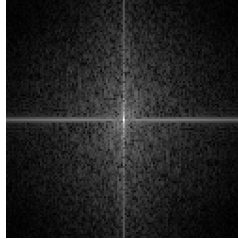
Sanity check

A vertical line in an image gets mapped to a **horizontal** line in FFT space!



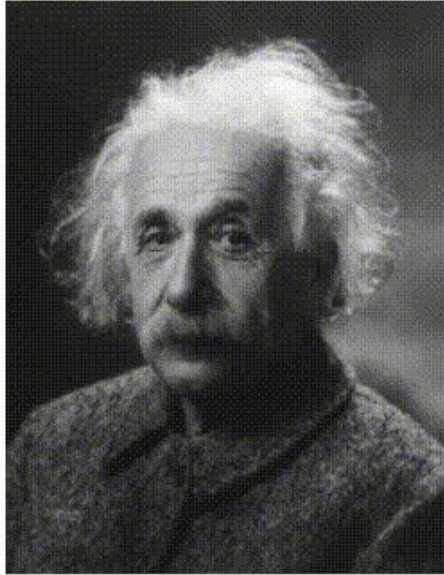
FFT for natural vs synthetic images

Real world is noisier, FFT plots look “grainier”

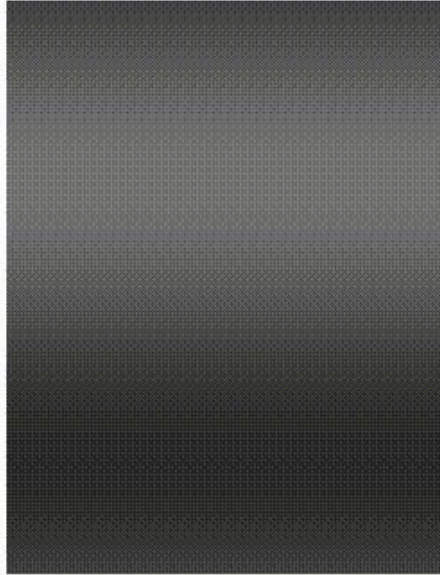


FFT can *perfectly* reconstruct an image!

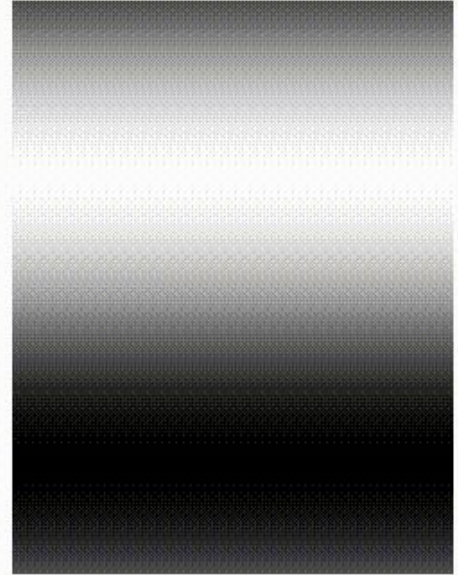
Original image



Reconstruction up to component 2

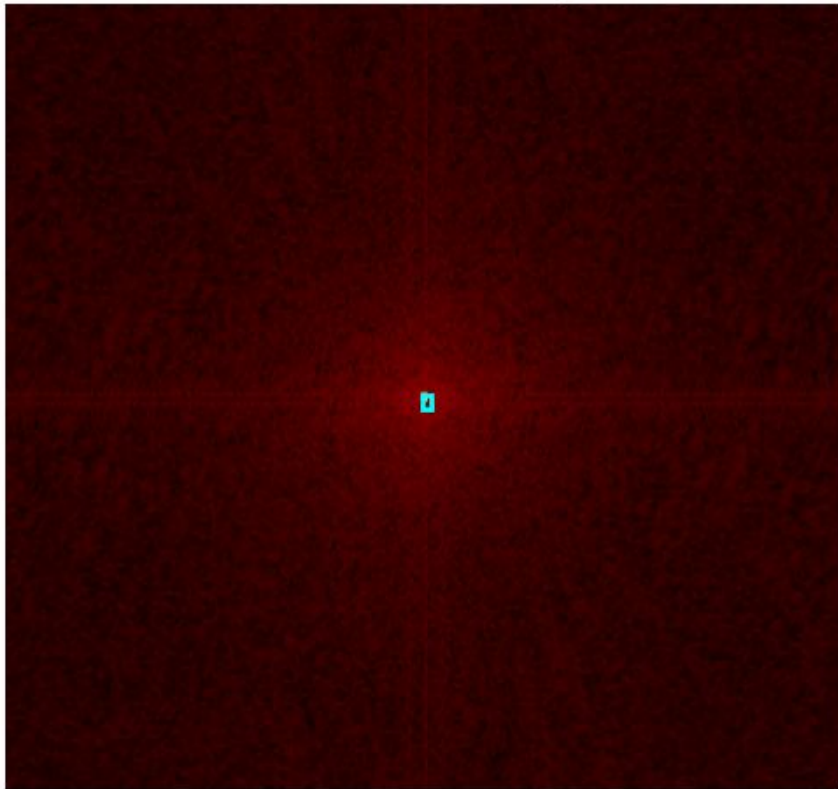


Current component (scaled)



FFT Masking

FFT with Mask (Size: 3px)



Reconstructed Image



Concept Check!

T/F: Convolution in the spatial domain is equivalent to multiplication in the frequency domain

Concept Check!

T/F: Convolution in the spatial domain is equivalent to multiplication in the frequency domain

True ("Convolution theorem")

Gaussian/Laplacian Stack

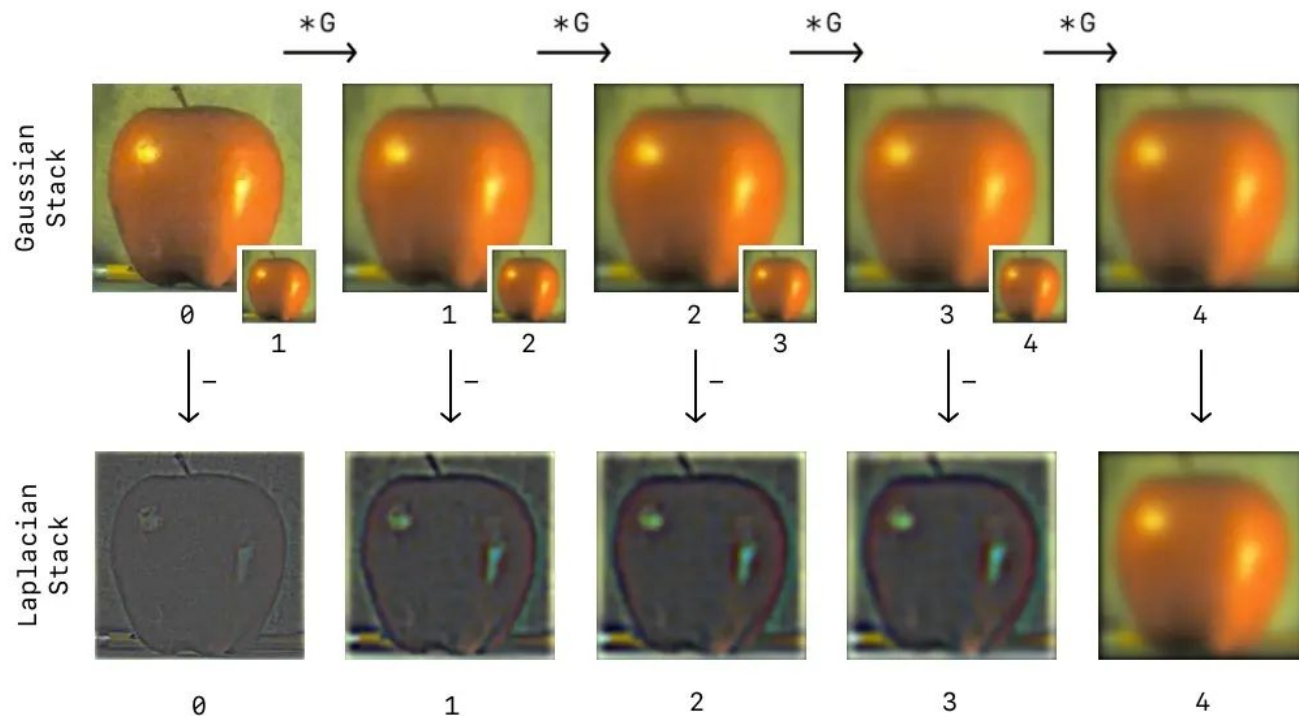
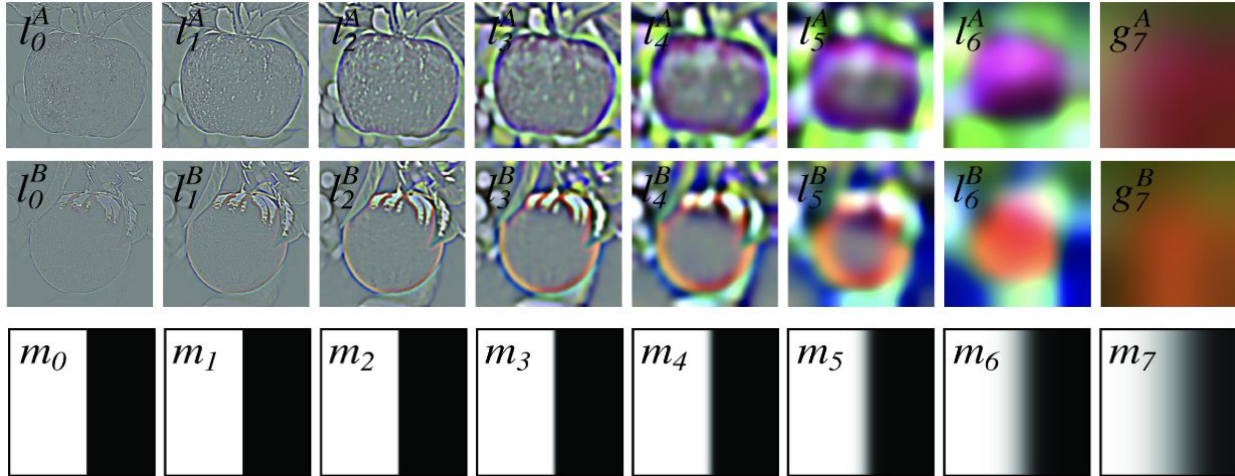


Image Blending

- We can use laplacian and gaussian stacks / pyramids to seamlessly blend two aligned images

$$\mathbf{l}_k = \mathbf{l}_k^A * \mathbf{m}_k + \mathbf{l}_k^B * (1 - \mathbf{m}_k)$$



Project 3A



Homogeneous Coordinates

Image Transformations

Homography

Interpolation

Homogeneous (Projective) Coordinates

- Coordinates of points, including *points at infinity*, can be represented using finite coordinates.
- Formulas involving homogeneous coordinates are often *simpler* and more *symmetric* than their Cartesian counterparts.

$$\begin{bmatrix} x \\ y \end{bmatrix} \xrightarrow{\text{homogeneous coords}} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

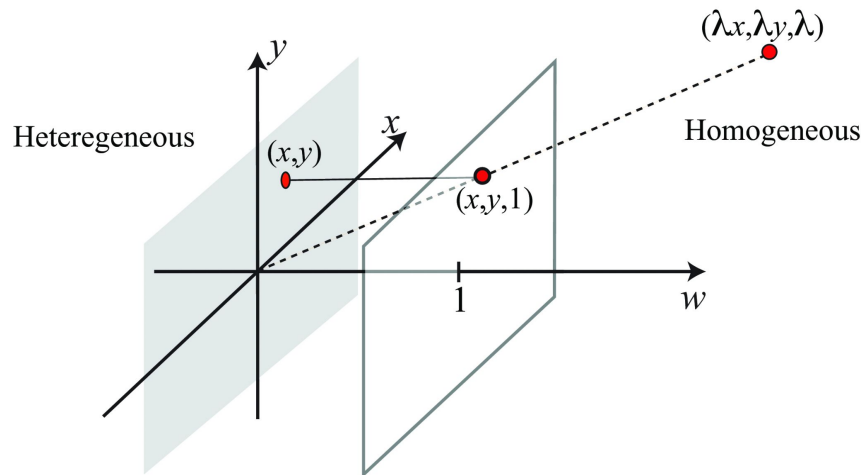
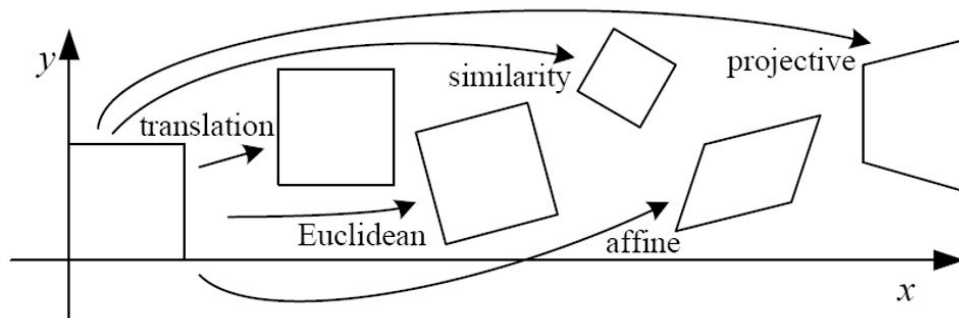
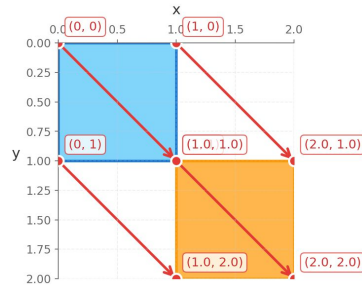


Image Transformations



Name	Matrix	# D.O.F.	Preserves:	Icon
translation	$\begin{bmatrix} I & t \end{bmatrix}_{2 \times 3}$	2	orientation + ...	
rigid (Euclidean)	$\begin{bmatrix} R & t \end{bmatrix}_{2 \times 3}$	3	lengths + ...	
similarity	$\begin{bmatrix} sR & t \end{bmatrix}_{2 \times 3}$	4	angles + ...	
affine	$\begin{bmatrix} A \end{bmatrix}_{2 \times 3}$	6	parallelism + ...	
projective	$\begin{bmatrix} \tilde{H} \end{bmatrix}_{3 \times 3}$	8	straight lines	

Image Transformations



Translation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Scale

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

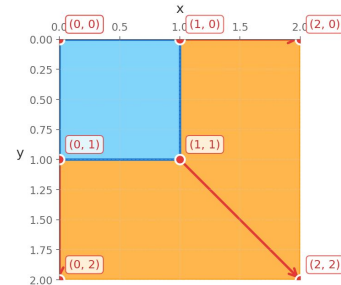
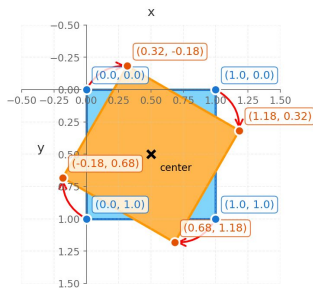


Image Transformations

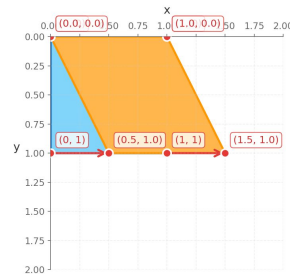


Rotation

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Shear

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



Chat with your neighbour!

T/F: An affine transformation can map parallel lines to intersecting lines.

T/F: A homography/projective transformation can bend straight lines into curves.

T/F: The order of transformations matters – applying rotation then translation generally gives a different result than applying translation then rotation.

Chat with your neighbour!

T/F: An affine transformation can map parallel lines to intersecting lines.

False, affine transformations preserve parallel lines

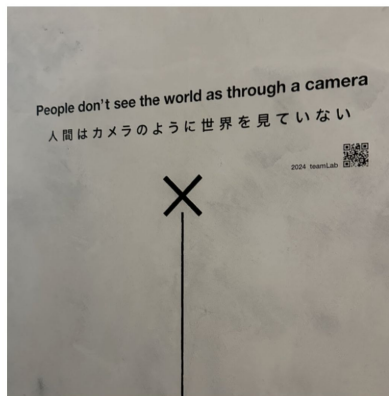
T/F: A homography/projective transformation can bend straight lines into curves.

False

T/F: The order of transformations matters – applying rotation then translation generally gives a different result than applying translation then rotation.

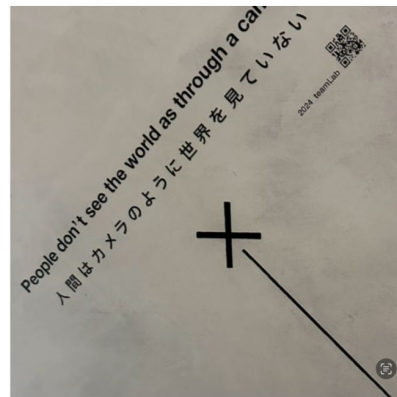
True

Homography

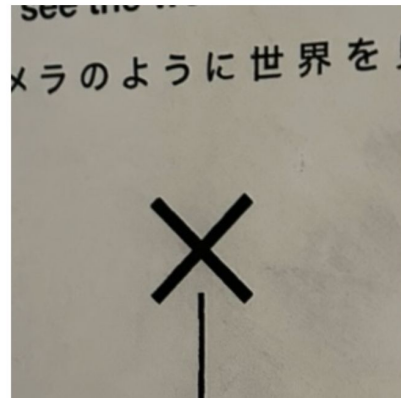


(Translation, Scale,
Rotation and Shear)

$$\begin{pmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix}$$



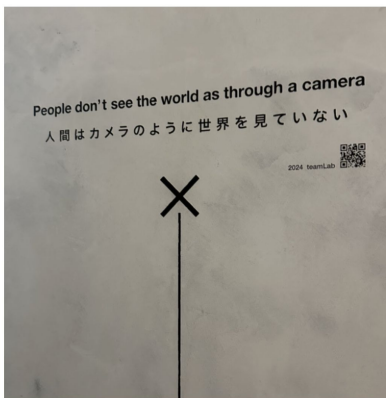
Rotation



Scale

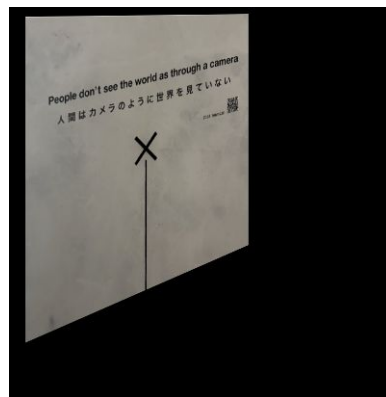
Homography

- Describes planar transformations – maps points on one plane to another.
- Encodes rotation, in-plane translation, scaling, shear and perspective distortion.
- Cannot represent parallax or nonlinear warps (e.g. lens distortion, fisheye effect or independent object motion).

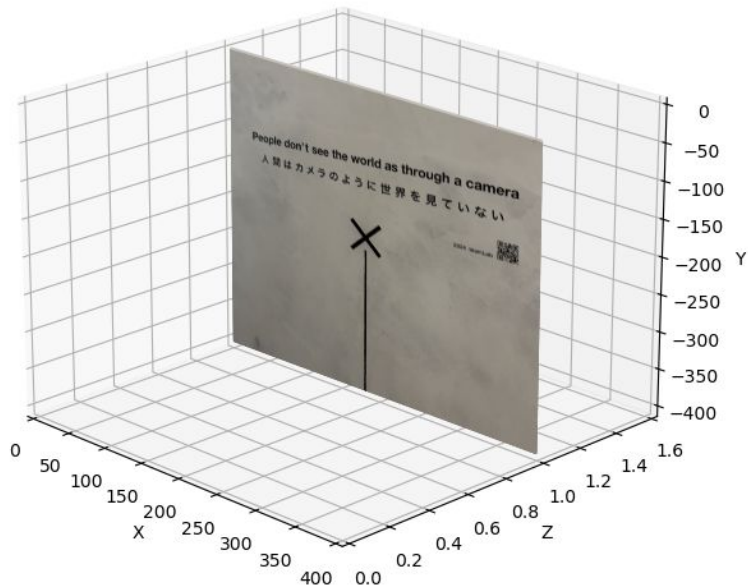


Homography

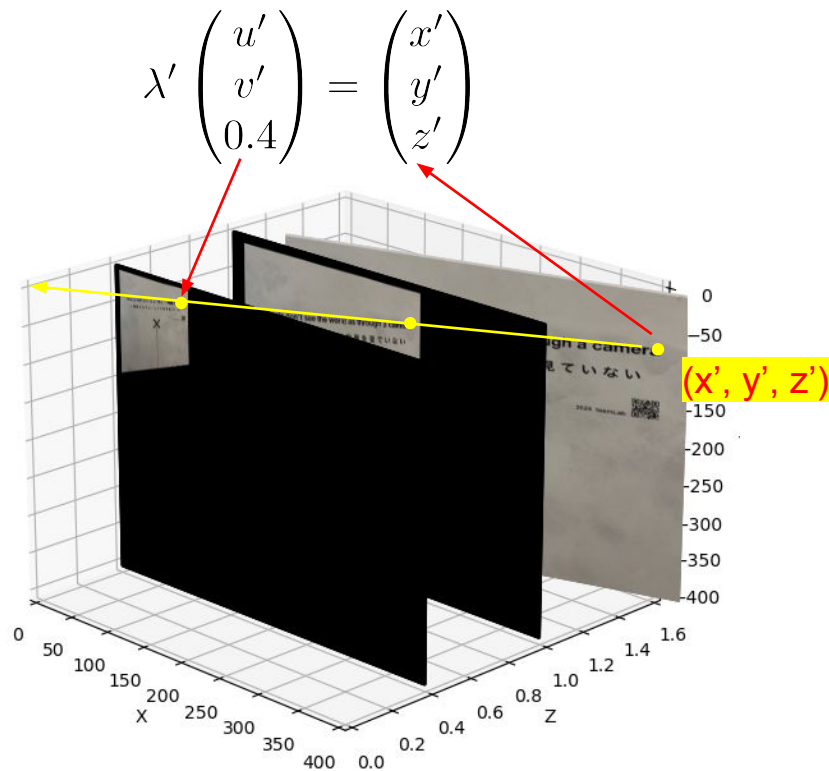
$$\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix}$$



Visualizing as planes...



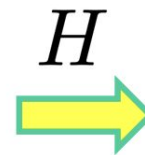
Perspective projected to $z = 0.4$ plane



How do we estimate it?

Correspondences!

+ Some linear algebra



Problem 2.2: Multiple Correspondences

(a) Suppose we have multiple correspondence pairs: $(x_1, y_1) \rightarrow (u_1, v_1)$, $(x_2, y_2) \rightarrow (u_2, v_2)$, $(x_3, y_3) \rightarrow (u_3, v_3)$, $(x_4, y_4) \rightarrow (u_4, v_4)$. Fill in the blanks in this matrix system for 4 correspondence pairs:

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -u_1x_1 & -u_1y_1 \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -v_1x_1 & -v_1y_1 \\ x_2 & y_2 & 1 & 0 & 0 & 0 & -u_2x_2 & -u_2y_2 \\ 0 & 0 & 0 & x_2 & y_2 & 1 & -v_2x_2 & -v_2y_2 \\ x_3 & y_3 & 1 & 0 & 0 & 0 & -u_3x_3 & -u_3y_3 \\ 0 & 0 & 0 & x_3 & y_3 & 1 & -v_3x_3 & -v_3y_3 \\ x_4 & y_4 & 1 & 0 & 0 & 0 & -u_4x_4 & -u_4y_4 \\ 0 & 0 & 0 & x_4 & y_4 & 1 & -v_4x_4 & -v_4y_4 \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \\ h_6 \\ h_7 \\ h_8 \end{bmatrix} = \begin{bmatrix} u_1 \\ v_1 \\ u_2 \\ v_2 \\ u_3 \\ v_3 \\ u_4 \\ v_4 \end{bmatrix}$$

Chat with your neighbour!

T/F: Homography relates two images from any camera that moves.

T/F: It is generally a good idea to keep the number of point correspondences at the minimum to avoid over-determining the homography.

How many DoF does a homography have?

Chat with your neighbour!

T/F: Homography relates two images from any camera that moves.

False! (perfect rotation is a special case which works)

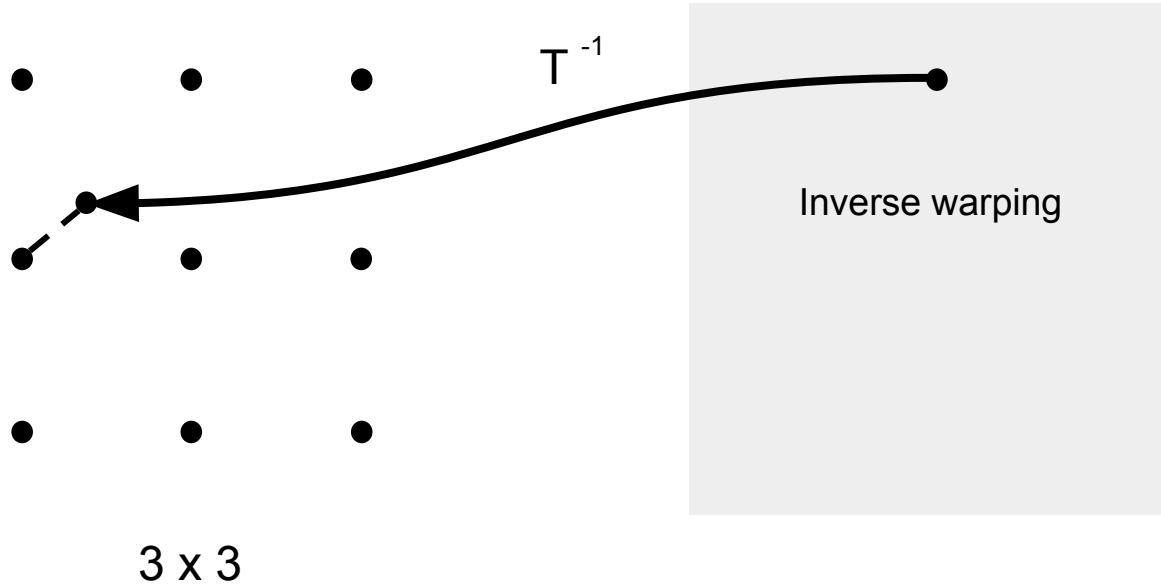
T/F: It is generally a good idea to keep the number of point correspondences at the minimum to avoid over-determining the homography.

False!

How many DoF does a homography have?

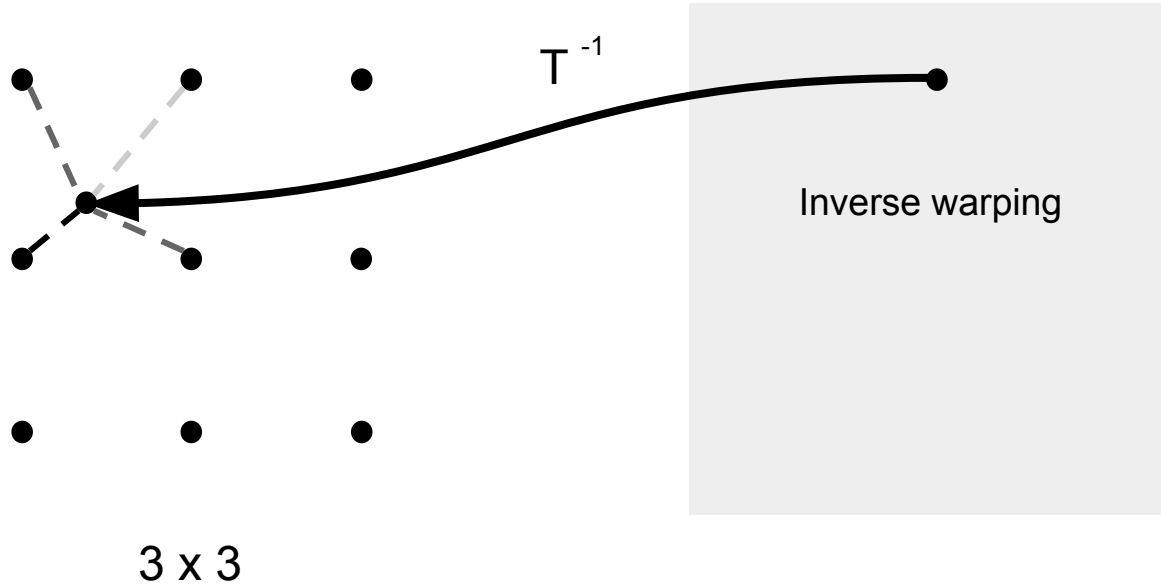
8

Nearest Neighbor Interpolation



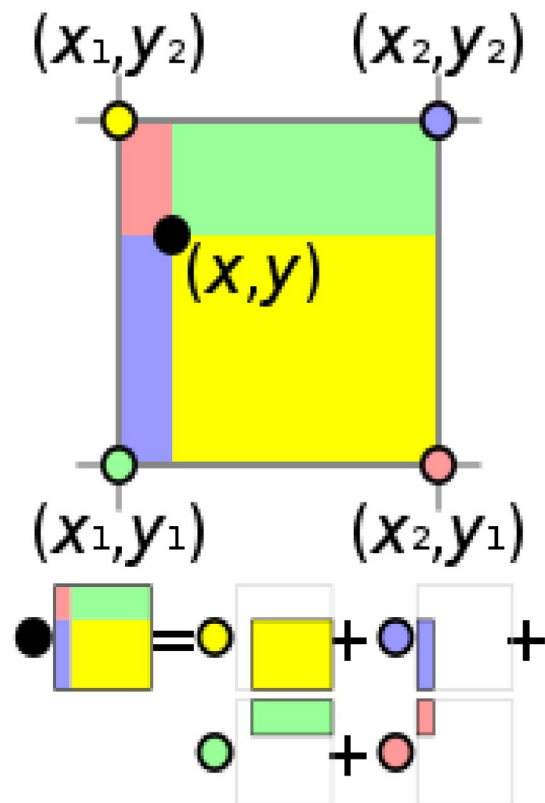
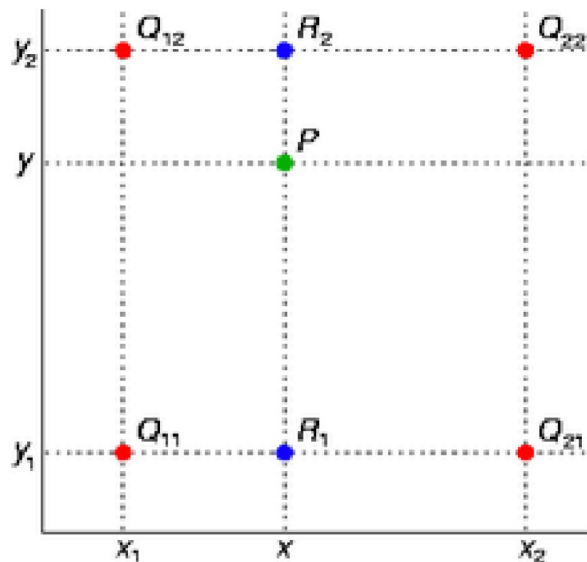
Pick closest (aka round)

Bilinear Interpolation



Weighted average of closest known points

Bilinear interp: use “the square”



Chat with your neighbour!

T/F: In inverse warping, bilinear interpolation computes a pixel's value using a weighted average of its four nearest source pixels.

T/F: Nearest-neighbor interpolation in inverse warping can produce smoother, less blocky images than bilinear interpolation.

Why do we iterate over the *output* pixels of transforms rather than input?

Chat with your neighbour!

T/F: In inverse warping, bilinear interpolation computes a pixel's value using a weighted average of its four nearest source pixels.

True

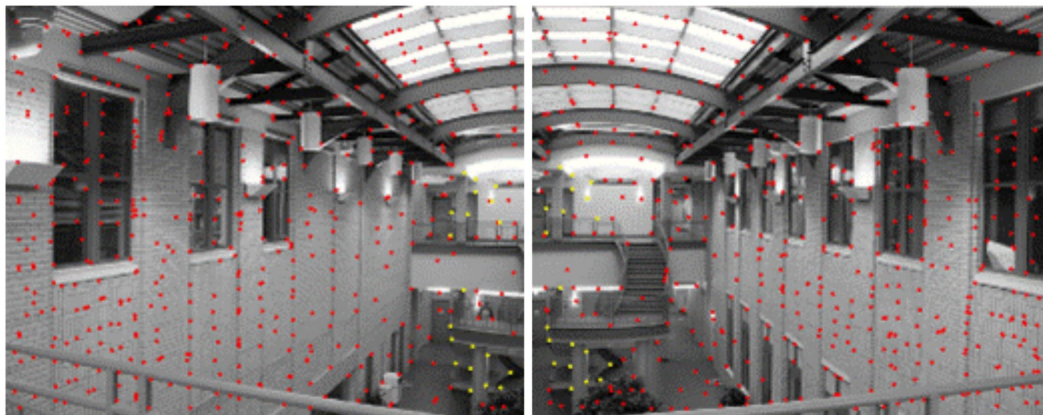
T/F: Nearest-neighbor interpolation in inverse warping can produce smoother, less blocky images than bilinear interpolation.

False

Why do we iterate over the *output* pixels of transforms rather than input?

Resolution problems; sometimes input can get mapped to many pixels of output. Easier to figure this out by inverting and sampling the input image

Project 3B



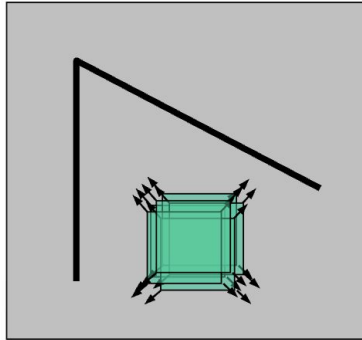
Feature Detection

RANSAC

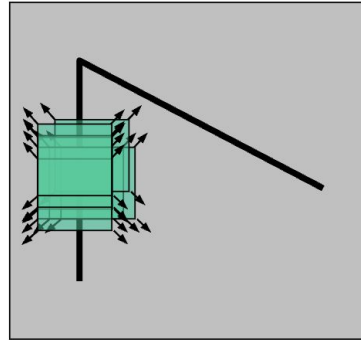
Feature Detection (Harris)

- We would like to determine interest points by detecting interesting features.
- So we measure changes along x and y directions.

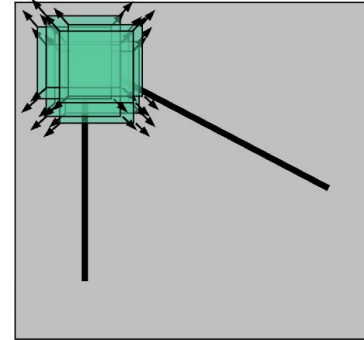
$$M = \sum_{(x,y) \in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$



No change



Change **except** in
edge direction



Change in **all**
directions

Feature Detection (Harris)

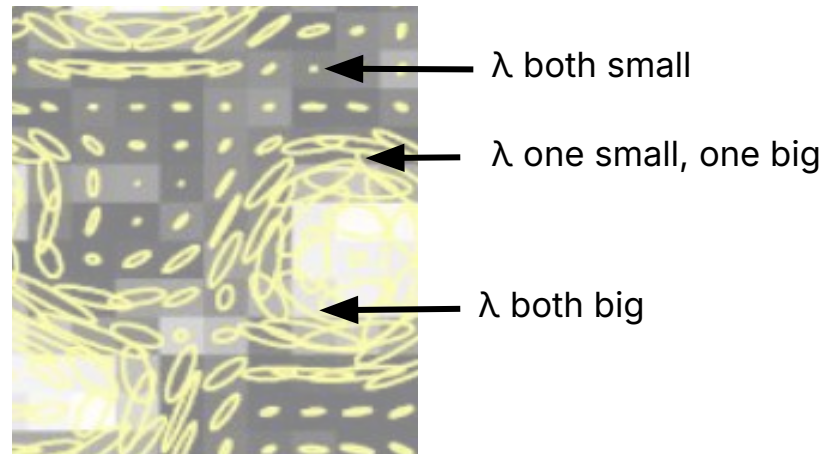
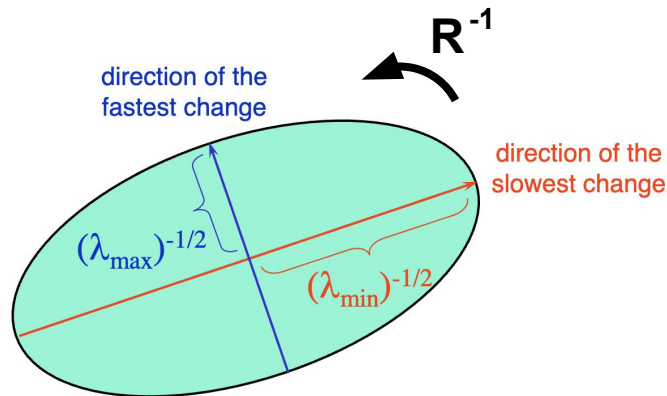
$$M = \sum_{(x,y) \in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

Second moment matrix (“structure tensor”)

One interpretation

$$M = R^{-1} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} R$$

Comes from SVD (we care about the eigenvalues)

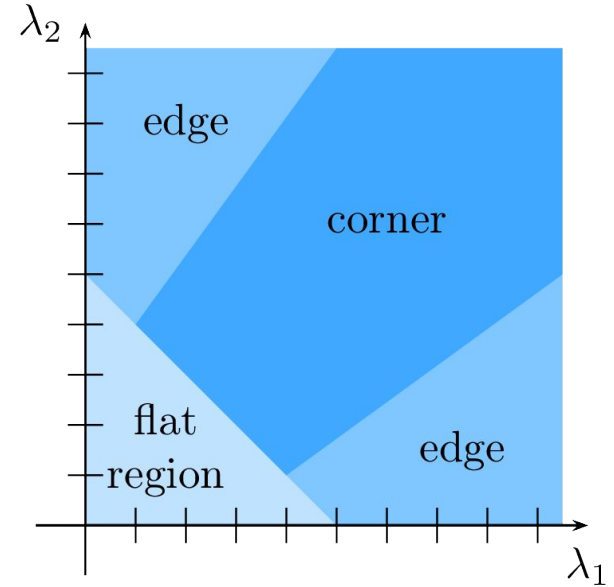


Feature Detection (Harris)

$$R = \frac{\det M}{\text{Trace } M} \quad R = \det(M) - k \cdot (\text{trace}(M))^2$$

$$\det M = \lambda_1 \lambda_2$$
$$\text{trace } M = \lambda_1 + \lambda_2$$

Product grows much faster than the sum
As λ grows, $\det(M) \gg \text{trace}(M)$



Chat!

Why do we care about corners in the first place?

True or false; Harris corner detection is perfectly invariant to 90-degree rotation

True or false; Corner detection matches the points between images

True or false; Corners are pixels with a lot of changes in either x or y directions.

Chat!

Why do we care about corners in the first place?

They give us good candidate points for descriptors, likely to be detected in 2 separate images and have unique appearance.

True or false; Harris corner detection is perfectly invariant to 90-degree rotation

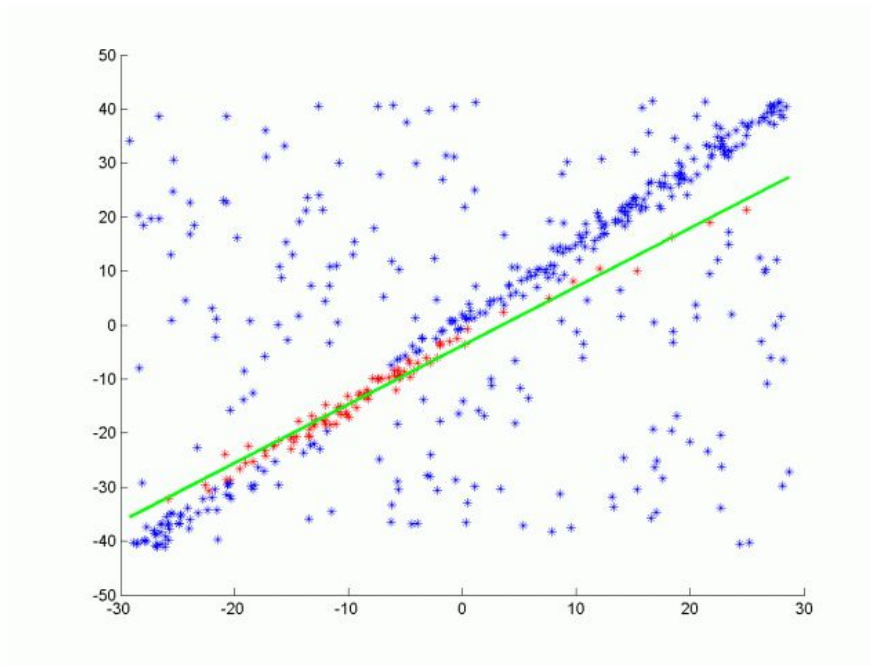
True or **false**; Corner detection matches the points between images

True or **false**; Corners are pixels with a lot of changes in either x or y directions.

Random Sample Consensus (RANSAC)

Why do we care? **Outliers** in correspondences make direct estimation impossible

1. Randomly sample *minimal correspondences* (4 for homography).
2. Estimate model (e.g. homography).
3. Compute inliers where error < threshold.
4. Keep model with most inliers and/or set of inliers.
5. Refit using all inliers (optional).



Chat!

T/F: RANSAC always picks N iterations = number of correspondences

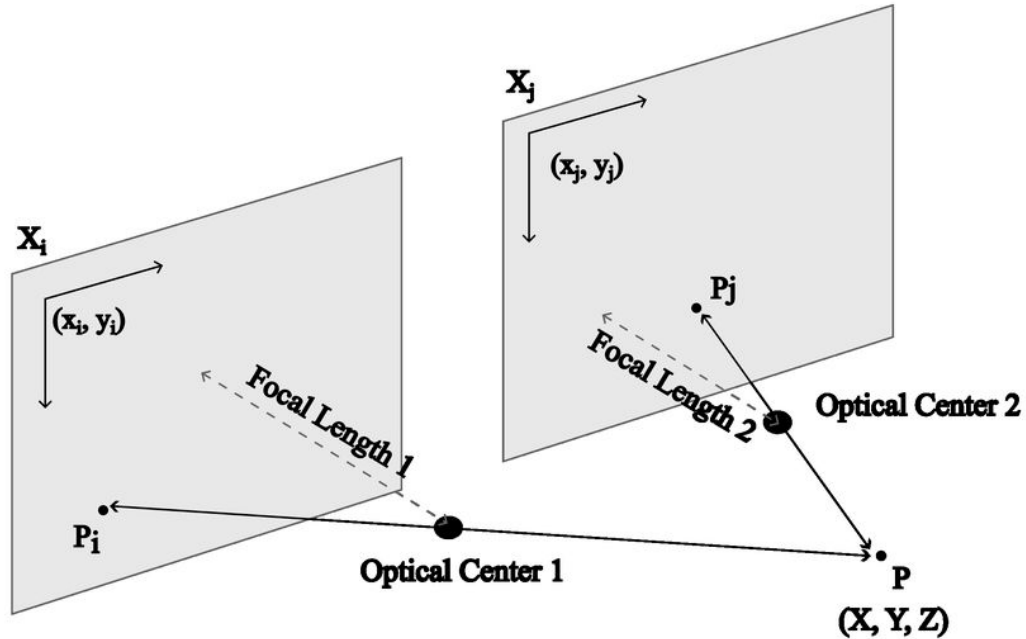
Chat!

T/F: RANSAC always picks N iterations = number of correspondences

False

Stereo

- Special case of 2 cameras:
 - **Parallel** optical axes, exactly the **same intrinsics**



Stereo

- 2 Main things to remember:
 - If you have pixel correspondence+camera distance, you get **depth!**
 - Correspondences lie on **horizontal lines**

